P2363R3
Extending associative containers with the remaining heterogeneous overloads

Published Proposal, 2022-01-19

This version:
http://wg21.link/P2363R3

Authors:
Konstantin Boyarinov (Intel)
Sergey Vinogradov (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The authors propose heterogeneous overloads for the remaining member functions in ordered and unordered associative containers.

Table of Contents

§ 1. Motivation

[N3657] and [P0919R0] introduced heterogeneous lookup support for ordered and unordered associative containers in C++ Standard Library. [P2077R2] proposed heterogeneous erasure overloads. As a result, a temporary key object is not created when a different (but comparable) type is provided as a key to the member function. But there are still no heterogeneous overloads for methods such as `insert_or_assign`, `operator[]` and others.

Consider the following example:

```
void foo(std::map<std::string, int, std::less<void>>& m) {
    const char* lookup_str = "Lookup_str";
    auto it = m.find(lookup_str); // matches to template overload
    // ...
    auto result = m.insert_or_assign(lookup_str, 1); // no heterogeneous alternative
}
```

Function `foo` accepts a reference to the `std::map<std::string, int>` object. A comparator for the map is `std::less<void>`, which provides `is_transparent` valid qualified-id, so the `std::map` allows using heterogeneous overloads with `Key` template parameter for lookup functions, such as `find`, `upped_bound`, `equal_range`, etc.

In the example above, the `m.find(lookup_str)` call does not create a temporary object of the `std::string` to perform a lookup. But the `m.insert_or_assign(lookup_str, 1)` call causes implicit conversion from `const char*` to `std::string` even if the insertion will not take place. The allocation and construction (as well as subsequent destruction and deallocation) of the temporary object of `std::string` *or any custom object* might be quite expensive and reduce the program performance.

All the proposed APIs in this paper allow to avoid mentioned performance penalty.

§ 2. Prior work

Heterogeneous lookup overloads for ordered and unordered associative containers were added into C++ standard. For example:

```
template <typename K>
iterator find(const K& x);
```

The corresponding overloads were added for `count`, `contains`, `equal_range`, `lower_bound` and `upper_bound` member functions.

[P2077R2] proposed heterogeneous erasure overloads for ordered and unordered associative containers:

```
template <typename K>
size_type erase(K&& x);

template <typename K>
node_type extract(K&& x);
```

Constraints:

- qualified-id `Compare::is_transparent` is valid and denotes a type for ordered associative containers
- qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types for unordered associative containers

where `Compare` is a type of comparator passed to an ordered container, `Hash` and `Pred` are types of hash function and key equality predicate passed to an unordered container respectively.

§ 3. Proposal overview

We propose to add heterogeneous overloads for the following member functions:

- `insert` in `std::set` and `std::unordered_set`
- `insert_or_assign`, `try_emplace`, `operator[]`, `at` in `std::map` and `std::unordered_map`
- `bucket` in `std::unordered_set`, `std::unordered_map`, `std::unordered_multiset` and `std::unordered_multimap`

§ 3.1. `try_emplace`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K, typename... Args>
std::pair<iterator, bool> try_emplace(K&& k, Args&&... args);

template <typename K, typename... Args>
iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

Effects: If the map already contains an element whose key compares equivalent with `k`, there is no effect. Otherwise, inserts an object of type `value_type` constructed with `std::piecewise_construct, std::forward_as_tuple(std::forward<K>(k)), std::forward_as_tuple(std::forward<Args>(args)...)`.

Constraints:

1. For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

2. `std::is_convertible_v<K&&, const_iterator>` and `std::is_convertible_v<K&&, iterator>` are both `false`.

Constraint 2 is introduced to maintain backward compatibility and makes homogeneous overload to be called when `K&&` is convertible to `const_iterator` or to `iterator`.

§ 3.2. `insert_or_assign`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K, typename M>
std::pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

template <typename K, typename M>
iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

Effects: If the map already contains an element `e` whose key compares equivalent with `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise, inserts an object of type `value_type` constructed with `std::forward<K>(k), std::forward<M>(obj)`.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

Backward compatibility problem in case of `K&&` convertibility to `iterator` or `const_iterator` will not take place because the number of arguments is fixed - call with three arguments would always consider the overloads with `hint` only.

§ 3.3. `operator[]`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K>
mapped_type& operator[](K&& k);
```

Effects: Equivalent to: `return try_emplace(std::forward<K>(k)).first->second`.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.4. `at`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K>
mapped_type& at(const K& k);

template <typename K>
const mapped_type& at(const K& k) const;
```

Returns: A reference to the `mapped_type` corresponding to `k` in `*this`.

Throws: An exception object of type `std::out_of_range` if no such element is present.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.5. `bucket`

We propose the following API (`std::unordered_map`, `std::unordered_multimap`, `std::unordered_set` and `std::unordered_multiset`):

```
template <typename K>
size_type bucket(const K& k) const;
```

Returns: The index of the bucket in which elements with keys compared equivalent with `k` would be found, if any such element existed.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.6. `insert`

We considered to add heterogeneous overloads of `insert` for all associative containers, but found the applicability only for `std::set` and `std::unordered_set` with the following API:

```
template <typename K>
std::pair<iterator, bool> insert(K&& k);

template <typename K>
iterator insert(const_iterator hint, K&& k);
```

Effects: If the set already contains an element which compares equivalent with `k`, there is no effect. Otherwise, inserts an object constructed with `std::forward<K>(k)` into the set.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

Adding heterogeneous `insert` overload makes no sence for associative containers with non-unique keys (`std::multimap`, `std::multiset`, `std::unordered_multimap` and `std::unordered_multiset`) because the insertion will be successful in any case and the key would be always constracted. All additional overheads introduced by `insert` can be mitigated by using `emplace`.

For `std::map` and `std::unordered_map`, heterogeneous insertion also makes no sence since heterogeneous `try_emplace` covers the relevant use cases.

§ 3.7. Design decisions

§ 3.7.1. Constructibility constraints

We do not introduce the following constraint: `std::is_constructible_v<value_type, K&&, Arguments>` for `try_emplace`, `insert_or_assign`, `operator[]` and `insert` member functions.

The only use-case when the homogeneous overload of the corresponding member function should be selected is the case when `K&&` is convertible to `key_type`, but `key_type` is not constructible from `K&&`. We decided not to consider such a use case as important to maintain.

The condition `std::is_constructible_v<value_type, K&&, Arguments>` is considered as a precondition for the corresponding member function.

§ 3.7.2. Insertion of implicitly convertible types

The proposed heterogeneous overloads for the insertion (such as `std::set::insert`) can result in adding heterogeneous keys that are only explicitly convertible to `key_type`.

Consider the following example:

```
std::set<std::shared_ptr<int>, std::owner_less<>> s;
std::weak_ptr<int> w = /*...*/;

s.insert(w);
```

The constructor of `std::shared_ptr` from the `std::weak_ptr` is `explicit`, so this code is currently ill-formed and the user should choose how to get `std::shared_ptr` from `w`. If the heterogeneous overload for `std::set::insert` would be added, the code above would become well-formed because implicit conversion is not required anymore and the insertion will happen via explicit conversion under the hood. If `w` is expired, the insertion will throw `std::bad_weak_ptr`.

We investigated adding constraints for heterogeneous overloads of the insertion methods so that they only participate in overload resolution if `K&&` is both implicitly and explicitly convertible to `key_type`, but it prohibits the use-case which is useful and common from authors perspective:

```
std::set<std::string, TransparentCharCompare> s;
std::string_view sv = /*...*/;

s.insert(sv);
```

The constructor of `std::string` from the `std::string_view` is explicit, so if heterogeneous overloads would be constrained as decribed above, the previous example would become ill-formed that forces the user to explicitly convert `sv` to `std::string`, which adds overhead (see § 3.9 Performance evaluation for more details).

Moreover, we found that the library already contains such precedences. First, we can compare the behavior of `std::vector::insert` and `std::vector::emplace`:

```
std::shared_ptr<int> sptr = /*...*/;
std::weak_ptr<int> wptr = sptr;

std::vector<std::shared_ptr<int>> v;
v.push_back(sptr); // OK
v.push_back(wptr); // Fail due to implicit conversion
v.emplace_back(wptr); // OK, insertion via explicit conversion
```

Another example is one of the overloads of `std::map::insert`:

```
template <class P>
std::pair<iterator, bool> insert(P&& x);
```

This overload is equivalent to `emplace(std::forward<P>(x))`. Hence, `insert(value_type&&)` and `insert(P&&)` are overloads with different semantics and the availability of such an overload results in the behavior described above. Consider:

```
std::shared_ptr<int> sptr;
std::weak_ptr<int> wptr;

std::pair<std::shared_ptr<int>, int> p1 = std::make_pair(sptr, 1); // OK
std::pair<std::shared_ptr<int>, int> p2 = std::make_pair(wptr, 1); // Fail

std::map<std::shared_ptr<int>, int, std::owner_less<>> m;
m.insert(std::make_pair(sptr, 1)); // OK
m.insert(std::make_pair(wptr, 1)); // Also OK, equivalent to emplace
```

Based on the described scenarios we came to the following conclusions:

- Do not introduce additional constraints to allow heterogeneous instertion for such types like `std::string_view` and `std::string` since we believe it is more common and important use-case.
- The implicit insertion possiblity of explicitly convertible types(as for `weak_ptr` and `shared_ptr`) into the containers already takes place in C++ standard library thus, doesn't look like showstopper for this proposal.

§ 3.7.3. Conversion and comparison consistency

The conversion from the heterogeneous key to `key_type` should be consistent with the heterogeneous lookup.

Consider the following example where inconsistency leads to unexpected result:

```
struct HeterogeneousKey {
  int value;
  operator int() const { return -value; }
};

struct Compare {
  using is_transparent = void;

  bool operator()(int lhs, int rhs) const {
    return lhs < rhs;
  }
  bool operator()(int lhs, HeterogeneousKey rhs) const {
    return lhs < rhs.value;
  }
  bool operator()(HeterogeneousKey lhs, int rhs) const {
    return lhs.value < rhs;
  }
};

int main() {
  std::set<int, Compare> s = {1, -2};
  s.insert(HeterogeneousKey{2});
}
```

In this case, `HeterogeneousKey{2}` will be compared with elements in the set as `2`, but would be inserted as `-2` and it results in broken `std::set` structure due to equivalent elements in the container.

Based on that, we need a precondition for heterogeneous insertion methods that the conversion from the heterogeneous key into `key_type` should produce an object that can be found with the heterogeneous lookup with the same heterogeneous key.

The example above also could result in breaking change. Currently the conversion from `HeterogeneousKey` to `int` is done before the insertion, so the value would be inserted only if the set does not contain elements that homogeneously compares equivalent to the converted value. If the heterogeneous overload for `insert` would be added, the conversion

would be done after the lookup phase inside the `insert` and the value would be inserted only if the set does not contain any elements that are heterogeneously compare equivalent to the `HeterogeneousKey`.

But we believe that the most common use-case for both heterogeneous lookup and insertion is the case when `key_type` and heterogeneous key represents the same thing like for `std::string` and `std::string_view`, which gives correct and unambiguous conversion from the heterogeneous key to `key_type`.

3.7.3.1. Inconsistency of comparison and conversion today

We would like to highlight weird behavior when heterogeneous comparison and conversion are inconsistent even without this proposal being accepted. Consider the following example:

```
struct HalfIs {
  int value;

  operator int() const { return value * 2; }
};

struct Compare {
  using is_transparent = void;

  bool operator()(int x, int y) const { return x < y; }
  bool operator()(int x, HalfIs y) const { return x < y / 2; }
  bool operator()(HalfIs x, int y) const { return x / 2 < y; }
};

int main() {
  std::set<int, Compare> s = {1, 2, 3, 5, 6};
  if (s.contains(HalfIs{2})) {
    bool result = s.insert(HalfIs{2}).second;
    assert(result == false); // Fails
    assert(s.size() == 5); // Fails
  }
}
```

In the example above heterogeneous lookup could potentially find two values (because of dividing by 2 in heterogeneous comparator overload) while `HalfIs::operator int` always produces the single value. `s.contains(HalfIs{2})` returns `true` because the element `5` compares equivalent with the value `HalfIs{2}` but if we try to add the same object into the container using `s.insert(HalfIs{2})` - the insertion will be successful due to conversion `HalfIs{2} -> 4`.

We think that such a behavior is questionable - why are we able to insert the element into the set if it already contains the equivalent one?

3.7.4. Use-cases with multiple matches

The design of heterogeneous lookup and erasure allows the unique-key associative containers to hold unique elements in terms of homogeneous comparisons, but potentially non-unique with heterogeneous comparisons:

```
struct Employee {
  // First component of the pair is the lastname,
  // the second is the firstname
  std::pair<std::string, std::string> fullname;

  Employee(const std::string& firstname, const std::string& lastname)
    : fullname(lastname, firstname) {}

  std::string firstname() const { return fullname.second; }
  std::string lastname() const { return fullname.first; }
};
```

```
struct Compare {
  using is_transparent = void;

  // Homogeneous comparison - compares both firstname and lastname
  bool operator()(const Employee& lhs, const Employee& rhs) const {
    return lhs.fullname < rhs.fullname;
  }

  // Heterogeneous comparisons - compare lastname and Employees lastname
  bool operator()(const Employee& lhs, const std::string& rhs) const {
    return lhs.lastname() < rhs;
  }

  // Reversed heterogeneous comparison
  bool operator()(const std::string& lhs, const Employee& rhs) const {
    return lhs < rhs.lastname();
  }
};

int main() {
  std::set<Employee, Compare> s = {{"John", "Smith"},
                                   {"Oliver", "Twist"},
                                   {"William", "Smith"}};

  // Below std::distance(r.first, r.second) == 2
  auto r = s.equal_range("Smith");
}
```

This code creates a `std::set` of employees that contains unique elements with homogeneous `Employee-Employee` comparison - combinations of the firstname and the lastname are unique. But with heterogeneous `Employee-lastname` comparison the set may contain multiple matches of the employees with the same lastname. It allows the user to find all Smiths in a set using single `equal_range("Smith")` call. The returned range will contain both John Smith and William Smith.

The reasonable question here is how should the heterogeneous insertion methods behave if multiple match occurred? In particular, `iterator` to which element should be returned? Or how many elements should be modified by calling `insert_or_assign`?

In general case we cannot not imagine how to write consistent conversion operator with the heterogeneous comparator that produces multiple match. If this scenario is possible then the answers could be that the returned iterator from the insertion should be defined in a consistent way with heterogeneous `find` where, in case of multiple matches, it returns the iterator to any mathed element. Similarly, the `insert_or_assign` assigns to any matched element.

Below we provide the ruminations why constistent converstion operator for heterogeneous key in the use-case with multiple match for unique-key containers is hardly possible.

To use heterogeneous insertion for the example above, we need to define the conversion from `std::string` to `Employee` to be able to emplace the value into the set:

```
struct Employee {
  // ...
  Employee(const std::string& lname);
  // ...
};
```

It's unclear how this constructor with one string (e.g., representing last name) would restore both first and last names. Thus, such construction does not seem meaningful. We think that for the use-case similar to above, such a conversion is considered as an attepts to restore the element using one of its properties with no attention to other properties. Similar examples are restoring the information about the car knowing that its color is red or restoring the integer using its hashcode.

One can argue that such a conversion can use a single string representing both first and last names, e.g. `"James Brown"` and fill the fullname by parsing the string. But that leads to the problem described in § 3.7.4 Use-cases with multiple matches section. Heterogeneous comparison should be written consistently to use the same parsing pattern, not just a single

lastname because otherwise, it (as it's written in the example) would compare last name with the string representing the full name, which leads to unpredictable results.

To conclude. We believe that there are three possible scenarios on the user side:

- There is heterogeneous comparison that could result in multiple match and there is no converion from heterogeneous key to `key_type`. In that case insertion (both homogeneous and heterogeneous) cannot be used and everything is just fine.
- There is heterogeneous comparison and a converion from heterogeneous key to `key_type` that are consistent with each other. In that case heterogeneous insertion produces the same effect on the container as homogeneous. Everything works as user expects.
- There is heterogeneous comparison and a converion from heterogeneous key to `key_type` that is **not** consistent with heterogeneous comparison. This scenario is covered by § 3.7.3 Conversion and comparison consistency and § 3.7.4 Use-cases with multiple matches sections. We suggest that it should be undefined behavior.

§ 3.7.5. `at` method design rationale

Despite that `at` method is put together with `operator[]` to "Element access" section in C++ standard, semantically it is much closer to `find` method because it does not make an insertion in case of element absence. Thus, we would like to specify `at` in terms of `find`. We also would like to add a precondition for `at` method that implies the same preconditions that `find` implicitly has.

In case of multiple match this method should return the reference to the same element as heterogeneous `find`.

§ 3.8. Proposed API summary

The proposed heterogeneous overloads for various methos are summarized in the table below:

Member function	std::set	std::multiset	std::map	std::multimap	std::unordered_set	std::unordered_multiset	std::unor
try_emplace	Not available	Not available	K&&	Not available	Not available	Not available	K&&
insert_or_assign	Not available	Not available	K&&	Not available	Not available	Not available	K&&
operator[]	Not available	Not available	K&&	Not available	Not available	Not available	K&&
at	Not available	Not available	const K&	Not available	Not available	Not available	const K&
bucket	Not available	Not available	Not available	Not available	const K&	const K&	const K&
insert	K&&	Not applicable	Not applicable	Not applicable	K&&	Not applicable	Not applic

§ 3.9. Performance evaluation

Our case study with the open-source *pmemkv* project confirms the importance of the case with `std::string` and `std::string_view` (see § 3.7.2 Insertion of implicitly convertible types for more details).

pmemkv is an embedded key/value data storage for persistent memory that provides several storage engines optimized for different use cases. We analyzed `vsmap` engine that is built on `std::map` data structure with `libmemkind::pmem::allocator` allocator for persistent memory from the *memkind library*. `std::basic_string` with the `libmemkind::pmem::allocator` is used as a key and value type.

```
using storage_type = std::basic_string<char, std::char_traits<char>,
                                       libmemkind::pmem::allocator<char>>;
using key_type = storage_type;
```

```
using mapped_type = storage_type;
using map_allocator_type = libmemkind::pmem::allocator<std::pair<const key_type,
                                                                 mapped_type>>;
using map_type = std::map<key_type, mapped_type, std::less<void>,
                          std::scoped_allocator_adaptor<map_allocator_type>>;
```

Initial implementation of the `vsmap::put` method was the following:

```
status vsmap::put(std::string_view key, std::string_view value)
{
  auto res = pmem_kv_container.try_emplace(key_type(key, kv_allocator), value);

  if (!res.second) {
    auto it = res.first;
    it->second.assign(value.data(), value.size());
  }

  return status::OK;
}
```

It explicitly creates a temporary object of `key_type` when the `try_emplace` method is called.

For performance evaluation, we extended LLVM Standard Library with the heterogeneous overload for the `try_emplace` method of `std::map`. With such overload the `vsmap::put` method does not need to create a temporary object of `key_type`:

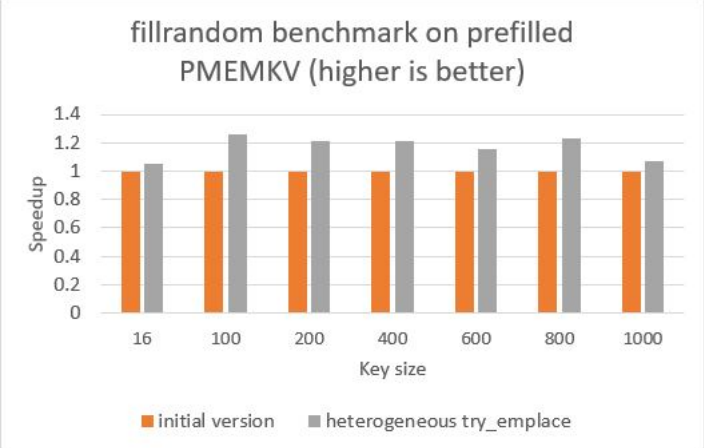
```
status vsmap::put(std::string_view key, std::string_view value)
{
  auto res = pmem_kv_container.try_emplace(key, value);

  if (!res.second) {
    auto it = res.first;
    it->second.assign(value.data(), value.size());
  }

  return status::OK;
}
```

For benchmarking we used the *pmemkv utility* and run the *fillrandom* benchmark on a prefilled storage (it means that the `vsmap::put` method updates existing element rather than inserting the new one). We executed the test with different key sizes (16, 100, 200, 400, 600, 800, 1000 bytes) and measured the throughput as operations per second.

The chart below shows performance increases relative to the initial implementation:



§ 4. Formal wording

Below, substitute the `&#xfffd;` character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 4.1. Modify class template `map` synopsis [**map.overview**]

> [...]
>
> ```
> // [map.access], element access
> mapped_type& operator[](const key_type& x);
> mapped_type& operator[](key_type&& x);
>
> template<class K> mapped_type& operator[](K&& x);
>
> mapped_type&         at(const key_type& x);
> const mapped_type&   at(const key_type& x) const;
>
> template<class K> mapped_type&       at(const K& x);
> template<class K> const mapped_type& at(const K& x) const;
> ```
>
> [...]
>
> ```
> template<class... Args>
>   pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
> template<class... Args>
>   pair<iterator, bool> try_emplace(key_type&& k, Args&&.. args);
>
> template<class K, class... Args>
>   pair<iterator, bool> try_emplace(K&& k, Args&&... args);
>
> template<class... Args>
>   iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
> template<class... Args>
>   iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
>
> template<class K, class... Args>
>   iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
>
> template<class M>
>   pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
> template<class M>
>   pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
>
> template<class K, class M>
>   pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
>
> template<class M>
>   iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
> template<class M>
>   iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
>
> template<class K, class M>
>   iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
> ```

§ 4.2. Modify [**map.access**]

[...]

```
  mapped_type& operator[](key_type&& x);
```

- *Effects*: Equivalent to: `return try_emplace(move(x)).first->second;`

```
  template<class K> mapped_type& operator[](K&& x);
```

- *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
- *Effects*: Equivalent to: `return try_emplace(forward<K>(x)).first->second;`

```
  mapped_type&       at(const key_type& x);
  const mapped_type& at(const key_type& x) const;
```

[...]

- *Complexity*: Logarithmic.

```
  template<class K> mapped_type&       at(const K& x);
  template<class K> const mapped_type& at(const K& x) const;
```

- *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
- *Preconditions*: The expression `find(x)` is well-formed and has well-defined behavior.
- *Returns*: A reference to `find(x)->second`.
- *Throws*: An exception object of type `out_of_range` if `find(x) == end()` is `true`.
- *Complexity*: Logarithmic.

§ 4.3. Modify [**map.modifiers**]

```
  template<class... Args>
    pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
  template<class... Args>
    iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class... Args>
    pair<iterator, bool> try_emplace(K&& k, Args&&... args);
  template<class K, class... Args>
    iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

— *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
For the first overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both `false`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `map` from `piecewise_construct, forward_as_tuple(forward<K>(k)), forward_as_tuple(forward<Args>(args)...)`.
The conversion from `k` into `key_type` constructs an object `u`, for which `find(k) == find(u)` is `true`.
— *Effects*: If the map already contains an element whose key is equivalent to `k`, there
Otherwise inserts an object of type `value_type` constructed with `piecewise_construct, forward_as_t` `forward_as_tuple(std::forward<Args>(args)...)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

[...]

```
  template<class M>
    pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
  template<class M>
    iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class M>
    pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
  template<class K, class M>
    iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

— *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
— *Mandates*: `is_assignable_v<mapped_type&, M&&>` is `true`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `map` from `forward<K>(k), forward<M>(obj)`.
The conversion from `k` into `key_type` constructs an object `u`, for which `find(k) == find(u)` is `true`.
— *Effects*: If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `std::forward<K>(k), std::forward<M>(obj)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

§ 4.4. Modify class template `set` synopsis [**set.overview**]

[...]

```
pair<iterator, bool> insert(const value_type& x);
pair<iterator, bool> insert(value_type&& x);

template<class K> pair<iterator, bool> insert(K&& x);

iterator insert(const_iterator hint, const value_type& x);
iterator insert(const_iterator hint, value_type&& x);

template<class K> iterator insert(const_iterator hint, K&& x);
```

§ 4.5. Add paragraph **Modifiers** into [**set.overview**]

Erasure [set.erasure]

[...]

Modifiers [set.modifiers]

```
template<class K> pair<iterator, bool> insert(K&& x);
template<class K> iterator insert(const_iterator hint, K&& x);
```

Constraints: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.

Preconditions: `value_type` is *Cpp17EmplaceConstructible* into `set` from `x`. The conversion from `x` into `key_type` constructs an object `u`, for which `find(x) == find(u)` is `true`.

Effects: Inserts a `value_type` object `t` constructed with `std::forward<K>(x)` if and only if there is no element in the container that is equivalent to

Returns: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the set element that is equivalent to `x`.

Complexity: Logarithmic.

§ 4.6. Modify [**tab.container.hash.req**]

Table **[illegible]**: **Unordered associative container requirements (in addition to container) [tab:container.hash.req]**

Expression	Return type	Assertion/ note pre- / post-condition	Complexity
[...]			
`b.bucket(k)`	`size_type`	*Preconditions*: `b.bucket_count() > 0`. *Returns*: The index of the bucket in which elements with keys equivalent to `k` would be found, if any such element existed. *Postconditions*: The return value shall be in the range `[0, b.bucket_count())`.	Constant
`a_tran.bucket(ke)`	`size_type`	*Preconditions*: `a_tran.bucket_count() > 0`. *Returns*: The index of the bucket in which elements with keys equivalent to `ke` would be found, if any such element existed. *Postconditions*: The return value shall be in the range `[0, a_tran.bucket_count())`.	Constant
[...]			

§ 4.7. Modify paragraph [illegible] in [**unord.req.general**]

[...]
The member function templates `find`, `count`, `equal_range`, ~~and~~ `contains` and `bucket` shall not participate in overload resolution unless the *qualified-ids* `Pred::is_transparent` and `Hash::is_transparent` are both valid and denote types ([illegible]).

§ 4.8. Modify class template `unordered_map` synopsis [**unord.map.overview**]

[...]

```
template<class... Args>
  pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
  pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);

template<class K, class... Args>
  pair<iterator, bool> try_emplace(K&& k, Args&&... args);

template<class... Args>
  iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
  iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);

template<class K, class... Args>
  iterator try_emplace(const_iterator hint, K&& k, Args&&... args);

template<class M>
  pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
  pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);

template<class K, class M>
  pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

template<class M>
  iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
  iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);

template<class K, class M>
  iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

[...]

```
// [unord.map.elem], element access
mapped_type& operator[](const key_type& k);
mapped_type& operator[](key_type&& k);

template<class K> mapped_type& operator[](K&& k);

mapped_type& at(const key_type& k);
const mapped_type& at(const key_type& k) const;

template<class K> mapped_type& at(const K& k);
template<class K> const mapped_type& at(const K& k) const;
```

[...]

```
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;

template<class K> size_type bucket(const K& k) const;
```

§ 4.9. Modify [**unord.map.access**]

[...]

```
  mapped_type& operator[](key_type&& k);
```

- *Effects*: Equivalent to: `return try_emplace(move(k)).first->second;`

```
  template<class K> mapped_type& operator[](K&& k);
```

- *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
- *Effects*: Equivalent to: `return try_emplace(forward<K>(k)).first->second;`

```
  mapped_type&       at(const key_type& k);
  const mapped_type& at(const key_type& k) const;
```

[...]

- *Throws*: An exception object of type `out_of_range` if no such element is present.

```
  template<class K> mapped_type&       at(const K& k);
  template<class K> const mapped_type& at(const K& k) const;
```

- *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
- *Preconditions*: The expression `find(k)` is well-formed and has well-defined behavior.
- *Returns*: A reference to `find(k)->second`.
- *Throws*: An exception object of type `out_of_range` if `find(k) == end()` is `true`.

§ 4.10. Modify [**unord.map.modifiers**]

```
  template<class... Args>
    pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
  template<class... Args>
    iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class... Args>
    pair<iterator, bool> try_emplace(K&& k, Args&&... args);
  template<class K, class... Args>
    iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

— *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
For the first overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both `false`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `piecewise_construct, forward_as_tuple(forward<K>(k)), forward_as_tuple(forward<Args>(args)...)`.
The conversion from `k` into `key_type` constructs an object `u`, for which `find(k) == find(u)` is `true`.
— *Effects*: If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct, forward_as_tuple(std::forward<K>(k)), forward_as_tuple(std::forward<Args>(args)...)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

[...]

```
  template<class M>
    pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
  template<class M>
    iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class M>
    pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
  template<class K, class M>
    iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

— *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
— *Mandates*: `is_assignable_v<mapped_type&, M&&>` is `true`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `forward<K>(k), forward<M>(obj)`.
The conversion from `k` into `key_type` constructs an object `u`, for which `find(k) == find(u)` is `true`.
— *Effects*: If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `std::forward<K>(k), std::forward<M>(obj)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

§ 4.11. Modify class template `unordered_multimap` synopsis [**unord.multimap.overview**]

[...]

```
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;

template<class K> size_type bucket(const K& k) const;
```

§ 4.12. Modify class template `unordered_set` synopsis [**unord.set.overview**]

[...]

```
pair<iterator, bool> insert(const value_type& obj);
pair<iterator, bool> insert(value_type&& obj);

template<class K> pair<iterator, bool> insert(K&& obj);

iterator insert(const_iterator hint, const value_type& obj);
iterator insert(const_iterator hint, value_type&& obj);

template<class K> iterator insert(const_iterator hint, K&& obj);
```

[...]

```
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;

template<class K> size_type bucket(const K& k) const;
```

§ 4.13. Add paragraph **Modifiers** into [**unord.set.overview**]

Erasure [unord.set.erasure]

[...]

Modifiers [unord.set.modifiers]

```
  template<class K> pair<iterator, bool> insert(K&& obj);
  template<class K> iterator insert(const_iterator hint, K&& obj);
```

— *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `unordered_set` from `x`. The conversion from `obj` into `key_type` constructs an object `u`, for which `find(obj) == find(u)` is `true`.
— *Effects*: Inserts a `value_type` object `t` constructed with `std::forward<K>(x)` if and only if there is no element in the container that is equivalent to
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the set element that is equivalent to `x`.
— *Complexity*: Average case - constant, worst case - linear.

§ 4.14. Modify class template `unordered_multiset` synopsis [**unord.multiset.overview**]

> [...]
>
> ```
> size_type bucket_size(size_type n) const;
> size_type bucket(const key_type& k) const;
>
> template<class K> size_type bucket(const K& k) const;
> ```

§ 5. Revision history

§ 5.1. R2 ➡ R3

- Added Preconditions for `at` method in § 4 Formal wording.
- Added § 3.7.5 at method design rationale.

§ 5.2. R1 ➡ R2

- Extended § 3.7 Design decisions to apply the feedback from LEWG.
- Added § 3.9 Performance evaluation.
- Updated § 4 Formal wording with the precondition about convertion and comparison consistency.

§ 5.3. R0 ➡ R1

- Removed `std::is_constructible_v` constraint. For more information see § 3.7 Design decisions.
- Added § 4 Formal wording.

§ 6. Acknowledgments

Special thanks to Tim Song for many findings during the paper review that eventually improved the quality of the proposal.

§ References

§ Informative References

[N3657]
J. Wakely, S. Lavavej, J. Muñoz. *Adding heterogeneous comparison lookup to associative containers (rev 4)*. 19 March 2013. URL: https://wg21.link/n3657

[P0919R0]
Mateusz Pusz. *Heterogeneous lookup for unordered containers*. 8 February 2018. URL: https://wg21.link/p0919r0

[P2077R2]
Konstantin Boyarinov, Sergey Vinogradov; Ruslan Arutyunyan. *Heterogeneous erasure overloads for associative containers*. 15 December 2020. URL: https://wg21.link/p2077r2